Mini Security Analytics Pipeline

Backend Assignment | Java / Spring Boot

Background

Imagine this: a coordinated cyberattack campaign, dubbed **"Operation Shadow Flood"**, is targeting major enterprises worldwide. Attackers are using a mix of SQL injection probes, credential stuffing bots, and slow-rate DDoS to overwhelm security teams with noise while exfiltrating data through the gaps. Security Operations Centers (SOCs) are drowning in millions of raw security events, with no way to see the big picture.

This is exactly the kind of problem that Akamai's **Web Security Analytics (WSA)** platform solves. WSA ingests millions of security event logs (called DLRs, short for Data Log Records) from Akamai's global edge network and processes them through a real-time pipeline that classifies attacks, enriches data, and computes threat scores. The platform exposes analytics through APIs that power dashboards and alerting systems. It's the tool that lets security analysts cut through the noise, spot attack patterns, and respond before damage is done.

> **Your assignment:** Build a simplified version of this system: a *mini security analytics pipeline* that ingests security events, processes and enriches them, stores them efficiently, and exposes analytics through a REST API. Think of it as building the backend brain of a SOC's war room dashboard.

Assignment Overview

Build a backend service called **"Mini WSA"** that:

1. **Ingests** security event records via a REST API.
2. **Processes** them by classifying, enriching, and aggregating the data.
3. **Stores** them in a storage engine of your choice.
4. **Exposes** analytics APIs for querying statistics and retrieving individual samples.

Domain Model

A **Security Event (DLR)** looks like this:

```
{
  "eventId": "evt-00132",
  "timestamp": "2026-05-20T14:32:10Z",
  "configId": 14227,
  "policyId": "pol_web1",
  "clientIp": "203.0.113.42",
  "hostname": "www.example.com",
  "path": "/api/v1/login",
  "method": "POST",
  "statusCode": 403,
  "userAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)",
  "rule": {
    "id": "950001",
    "name": "SQL_INJECTION",
    "message": "SQL Injection Attack Detected",
    "severity": "CRITICAL",
    "category": "INJECTION"
  },
  "action": "DENY",
  "geoLocation": {
    "country": "CN",
    "city": "Beijing"
  },
  "requestSize": 1024,
  "responseSize": 256
}
```

The `rule.category` field can be one of:

- `INJECTION` (SQL injection, command injection, etc.)
- `XSS` (cross-site scripting)
- `PROTOCOL_VIOLATION` (malformed HTTP)
- `DATA_LEAKAGE` (sensitive data exposure)
- `BOT` (bot / scraper activity)
- `DOS` (denial of service patterns)
- `RATE_LIMIT` (rate limiting triggered)

The `action` field can be: `DENY`, `ALERT`, or `MONITOR`.

Requirements

Part 1: Ingestion API

Create an endpoint to receive security events:

```
POST /v1/events/ingest
```

- Accepts a single event or a batch of events (array).
- Validates incoming events against the schema (required fields, valid enum values, valid timestamp format).
- Returns appropriate HTTP status codes (201 for success, 400 for validation errors with details).
- Assigns a server-side `receivedAt` timestamp to each event on ingestion.

Part 2: Classification & Enrichment

When an event is ingested, the service must:

1. Classify the attack type: map `rule.category` to a human-readable `attackType` string:

rule.category	attackType
`INJECTION`	SQL/Command Injection
`XSS`	Cross-Site Scripting
`PROTOCOL_VIOLATION`	Protocol Anomaly
`DATA_LEAKAGE`	Data Exfiltration
`BOT`	Bot Activity
`DOS`	Denial of Service
`RATE_LIMIT`	Rate Limiting

2. Compute a threat score (integer 0-100) based on:

- `rule.severity`: CRITICAL = 40, HIGH = 30, MEDIUM = 20, LOW = 10
- `action`: DENY adds +20, ALERT adds +10, MONITOR adds +0
- If `path` contains `/admin` or `/login`, add +15
- If more than 5 events from the same `clientIp` exist within the last 10 minutes, add +15 (repeat offender bonus)

- Cap at 100.

3. Store the enriched event with the original fields plus `attackType`, `threatScore`, and `receivedAt`.

Part 3: Statistics API

```
GET /v1/stats/summary?configId={configId}&from={ISO8601}&to={ISO8601}
```

Returns aggregated statistics for a given configuration and time range:

```
{
  "configId": 14227,
  "timeRange": { "from": "...", "to": "..." },
  "totalEvents": 1523,
  "byCategory": {
    "INJECTION": { "count": 450, "avgThreatScore": 72.3 },
    "BOT": { "count": 380, "avgThreatScore": 45.1 }
  },
  "byAction": {
    "DENY": 890,
    "ALERT": 433,
    "MONITOR": 200
  },
  "topAttackers": [
    { "clientIp": "203.0.113.42", "count": 87, "avgThreatScore": 81.2 }
  ],
  "topTargetedPaths": [
    { "path": "/api/v1/login", "count": 234 }
  ]
}
```

- `topAttackers` returns the top 10 by event count.
- `topTargetedPaths` returns the top 10 by event count.
- If `configId` is omitted, aggregate across all configurations.

Part 4: Samples API

```
GET /v1/events/samples?configId={configId}&from={ISO8601}&to=
{ISO8601}&category={category}&action={action}&limit={limit}&offset={offset}
```

- Returns individual enriched event records matching the filters.
- Supports pagination via `limit` (default 20, max 100) and `offset`.

- All filter parameters are optional.
- Results are sorted by `timestamp` descending (newest first).
- Response includes a `total` count of matching events (for pagination).

Part 5: Data Generation

Create a **data generator** script or module that:

- Generates realistic-looking random security events.
- Simulates "attack waves" (bursts of events from the same IP hitting the same path).
- Can generate a configurable number of events (e.g., 10,000).
- Outputs events in a format that can be fed into the ingestion API.

> This is important so that you (and we) can test the system with meaningful data.

Technical Requirements

- **Language:** Java with Spring Boot for the REST APIs.
- **Storage:** Any storage engine you choose. Keep in mind that in the real world, this system handles **big data scale**. Justify your choice in the README.
- **Build & Run:** Provide clear instructions on how to build and run your project. A `docker-compose up` setup that brings up the full stack (app + dependencies) is a **bonus**, but not required. At a minimum, we should be able to build and run your project by following your README instructions.
- **Testing:** Include unit tests for the classification and enrichment logic, plus at least one integration test for the API.
- **API Documentation:** Provide a README with clear setup instructions, API documentation (curl examples are fine), and an architecture description.

Submission

1. Create a **public GitHub repository** for this assignment.
2. Use meaningful commit messages. We want to see how you break down the work, so make small, logical commits (not one giant commit at the end).
3. Tag milestones as you go: `v0.1-ingestion`, `v0.2-enrichment`, `v0.3-stats`, etc.
4. Include a `README.md` with:
    - How to build and run the project.

- Architecture overview (a simple diagram is a plus).
- Storage choice justification.
- What you would improve if you had more time.
- Which parts you found challenging and how you solved them.

5. Email the GitHub repository details to **obokobza@akamai.com** a day before the technical interview.

Bonus Challenges

Choose one of the following challenges and implement them to demonstrate extra depth (Optional):

1. **Alerting Endpoint:** `POST /v1/alerts/define` and `GET /v1/alerts/evaluate`
   - Allow defining simple alert rules: "If more than N events of category X happen within Y minutes, trigger an alert."
   - The evaluate endpoint checks current data against all defined rules and returns which alerts are currently firing.
2. **Time-Series Dashboard Data:** Add an endpoint:

   ```
   GET /v1/stats/timeseries?configId={configId}&from={}&to={}&interval={1m|5m|1h}
   ```

   Returns event counts bucketed by the specified time interval, suitable for drawing a line chart.
3. **Rate Limiting:** Add rate limiting to the stats and samples APIs (e.g., 100 requests per minute per client IP). Return `429 Too Many Requests` when exceeded.

Notes

- You **may** use AI tools (Claude, ChatGPT, Copilot, etc.). We consider this a normal part of modern development. However, you must **understand everything in your code**. During the interview, we will ask you to explain specific implementation details, modify the code live, and discuss alternatives.
- Focus on **working software** over perfection. A running system with 80% of the features beats a non-running system with 100%.
- If you get stuck on something, document what you tried and move on. We value problem-solving approach over completeness.

Questions?

Please forward any questions to **obokobza@akamai.com**.

Good luck!

The Akamai CSI Team